

# CPI Hub: Extremely Detailed Technical Product Requirements Document

## 1. System Overview

### 1.1 Product Definition

CPI Hub is an intelligent collaboration platform that aggregates data from multiple sources, applies AI-powered contextual analysis, provides collaborative workspaces, enables advanced search capabilities, and enforces role-based access controls.

### 1.2 System Architecture

- **Frontend:** React.js with TypeScript, Redux for state management
- **Backend:** Node.js with Express, GraphQL API
- **Database:** Primary - Firestore, Secondary - PostgreSQL for relational data
- **Storage:** Cloud Storage (Google Cloud Storage or AWS S3)
- **AI Services:** TensorFlow.js for frontend analysis, Python with scikit-learn/spaCy/NLTK for backend processing
- **Authentication:** OAuth 2.0, JWT tokens
- **Deployment:** Docker containers orchestrated with Kubernetes
- **CI/CD:** GitHub Actions with automated testing and deployment

### 1.3 Technical Dependencies

- Google Drive API v3
- Slack API (Web API and Events API)
- Gmail API and Microsoft Graph API for Outlook
- OAuth 2.0 authentication providers
- NLP libraries (spaCy, NLTK, TensorFlow)
- D3.js for visualization
- Elasticsearch for search functionality

## 2. Data Aggregation and Integration Specifications

### 2.1 Authentication Subsystem

### 2.1.1 OAuth Integration

- **Implementation:** OAuth 2.0 with PKCE (Proof Key for Code Exchange)
- **Supported Providers:**
  - Google (Drive, Gmail)
  - Microsoft (Outlook)
  - Slack
- **Token Storage:**
  - Encryption: AES-256 for token encryption at rest
  - Storage: Secure token storage in Firestore with encrypted fields
  - Refresh Mechanism: Automatic token refresh when expiration is <24 hours away

### 2.1.2 Authentication Workflow

- **User Flow:**
  1. User initiates connection to service
  2. System redirects to OAuth provider
  3. User authenticates with provider
  4. Provider redirects to CPI Hub with authorization code
  5. Backend exchanges code for access and refresh tokens
  6. Tokens are encrypted and stored
- **Revocation Flow:**
  1. User initiates disconnection
  2. System calls provider API to revoke tokens
  3. System deletes stored tokens
  4. System logs revocation event

### 2.1.3 Technical Requirements

- Implement token rotation every 7 days
- Maximum token validity period: 30 days
- Implement CSRF protection for OAuth callbacks
- Maintain audit log of all authentication events

## 2.2 Data Fetching Subsystem

### 2.2.1 Fetch Scheduling

- **Scheduler Implementation:**
  - Technology: node-cron for scheduled tasks
  - Default Intervals:
    - High priority sources: Hourly
    - Medium priority sources: Every 6 hours
    - Low priority sources: Daily
  - Admin Configuration Panel:
    - UI for customizing fetch intervals per source type

- Immediate manual trigger option
- Fetch history logs

### 2.2.2 API Client Implementation

- **Google Drive Client:**

- Endpoints:
  - `/files` - List all files and folders
  - `/files/{id}` - Get specific file metadata
  - `/files/{id}/content` - Download file content
- Rate Limiting:
  - Implement exponential backoff (starting with 1s, max 5 minutes)
  - Maintain usage quotas per user account
  - Implement request batching for efficiency
- Error Handling:
  - Log API errors with full context
  - Categorize errors (auth, rate limit, server)
  - Implement retry logic with max 3 attempts

- **Slack Client:**

- Endpoints:
  - `conversations.list` - Get all channels/DMs
  - `conversations.history` - Get messages
  - `files.list` - Get file attachments
  - `files.info` - Get file metadata
- Rate Limiting:
  - Implement tier-based rate limiting respecting Slack API policies
  - Queue requests during rate limit periods
- Error Handling:
  - Log workspace-specific errors
  - Handle workspace migrations
  - Implement retry logic excluding permanent errors

- **Email Client:**

- Gmail API:
  - `users.messages.list` - List messages
  - `users.messages.get` - Get message content
  - `users.messages.attachments.get` - Get attachments
- Microsoft Graph API:
  - `/messages` - List messages
  - `/messages/{id}` - Get message content
  - `/messages/{id}/attachments` - Get attachments

- Rate Limiting:
  - Implement provider-specific rate limit handling
  - User-specific quotas per email account
- Error Handling:
  - Account-specific error logging
  - Categorize by provider-specific error codes
  - Implement retry strategy appropriate for each provider

### 2.2.3 Incremental Data Fetching

- **Implementation:**
  - Store `lastSyncTimestamp` per data source
  - Use API-specific delta/change tokens where available
  - For Google Drive:
    - Use `changes.list` with `pageToken` for incremental sync
  - For Slack:
    - Store timestamp of last message fetched per channel
  - For Email:
    - Store ID of last message fetched
  - Implement change detection for modified files
- **Data Consistency:**
  - Maintain hash values of fetched content
  - Compare hashes to detect content changes
  - Implement conflict resolution for simultaneous updates

## 2.3 Data Extraction Subsystem

### 2.3.1 Document Processing

- **Text Extraction:**
  - Google Docs:
    - Use Google Drive Export API to convert to plain text
    - Preserve document structure (headers, lists, tables)
    - Extract comments and suggestions
  - PDFs:
    - Use `pdf.js` for text layer extraction
    - Implement OCR (`Tesseract.js`) for scanned PDFs
    - Extract metadata (title, author, creation date)
  - Microsoft Office:
    - Convert to HTML using appropriate libraries
    - Extract text and formatting from HTML
- **Tabular Data:**
  - Google Sheets:
    - Use Sheets API to fetch structured data
    - Preserve sheet structure, formulas, and formatting

- Handle cell formulas and references
  - Excel:
    - Extract tables using xlsx library
    - Preserve cell formats and data types
    - Extract named ranges and pivot tables
- **Metadata Extraction:**
  - Common Fields:
    - Title, Author, Created Date, Modified Date, File Size, MIME Type
  - Extended Fields:
    - Version History, Permission Settings, Parent Folders
    - Custom Properties, Tags

### 2.3.2 Message Processing

- **Slack Processing:**
  - Message Text:
    - Extract raw and formatted text
    - Process markdown and Slack-specific formatting
    - Extract mentions, emojis, and reactions
  - Thread Processing:
    - Maintain parent-child relationship of threads
    - Link replies to original messages
  - File Attachments:
    - Download and process attachments
    - Extract preview images
    - Link to original file in Slack
- **Email Processing:**
  - Body Extraction:
    - Extract both plain text and HTML versions
    - Convert HTML to markdown for consistent storage
    - Handle inline images and formatting
  - Header Processing:
    - Extract From, To, CC, BCC, Subject, Date
    - Process Reply-To and References headers
    - Handle email threads and conversations
  - Attachment Processing:
    - Download attachments up to 25MB size limit
    - Generate previews for common formats
    - Extract text from attachments where possible

### 2.3.3 Data Normalization

- **Text Normalization:**

- Unicode normalization (NFC form)
  - Consistent newline handling (LF)
  - Character encoding standardization (UTF-8)
- **Date Normalization:**
  - Convert all timestamps to ISO 8601 format
  - Store original timezone information
  - Maintain UTC timestamps in database
- **Entity Recognition:**
  - Identify and normalize person names across platforms
  - Map email addresses to user accounts
  - Correlate identities across platforms

## 2.4 Data Storage Subsystem

### 2.4.1 Database Schema

- **Core Collections:**
  - `users` - User accounts and preferences
  - `dataConnections` - Integration auth details
  - `dataSources` - Source configuration and metadata
  - `documents` - Document metadata and extracted content
  - `messages` - Chat/email message content and metadata
  - `attachments` - File attachment metadata
  - `projects` - Project workspaces configuration
  - `permissions` - Access control definitions
- **Document Schema:**
  - {
  - "id": "string",
  - "sourceType": "enum(GDRIVE|SLACK|EMAIL)",
  - "sourceId": "string",
  - "title": "string",
  - "content": "string",
  - "contentType": "string",
  - "mimeType": "string",
  - "metadata": {
  - "author": "string",
  - "createdAt": "ISO8601 timestamp",
  - "modifiedAt": "ISO8601 timestamp",
  - "parentPath": "string",
  - "version": "number",
  - "size": "number"
  - },
  - "extracted": {

- "keywords": ["string"],
- "topics": ["string"],
- "entities": [{
- "type": "enum(PERSON|ORG|LOCATION|DATE)",
- "value": "string",
- "positions": [[start, end]]
- }],
- "summary": "string"
- },
- "attachments": ["string (attachment\_id)"],
- "permissions": ["string (permission\_id)"],
- "deleted": "boolean",
- "lastSyncedAt": "ISO8601 timestamp"
- }

● **Message Schema:**

- {
- "id": "string",
- "sourceType": "enum(SLACK|EMAIL)",
- "sourceId": "string",
- "channelId": "string",
- "threadId": "string",
- "sender": {
- "id": "string",
- "name": "string",
- "email": "string"
- },
- "recipients": [{
- "id": "string",
- "name": "string",
- "email": "string",
- "type": "enum(TO|CC|BCC)"
- }],
- "content": "string",
- "contentFormatted": "string",
- "subject": "string",
- "timestamp": "ISO8601 timestamp",
- "attachments": ["string (attachment\_id)"],
- "reactions": [{
- "emoji": "string",
- "count": "number",
- "users": ["string (user\_id)"]
- }],

- "extracted": {
- "keywords": ["string"],
- "entities": [{
- "type": "enum(PERSON|ORG|LOCATION|DATE)",
- "value": "string",
- "positions": [[start, end]]
- }]
- },
- "metadata": {
- "labels": ["string"],
- "importance": "enum(HIGH|NORMAL|LOW)"
- },
- "hasBeenRead": "boolean",
- "permissions": ["string (permission\_id)"]
- }

### 2.4.2 File Storage

- **Implementation:**
  - Use Google Cloud Storage or AWS S3 for file storage
  - Folder structure:
    - `{tenant_id}/{source_type}/{yyyy-mm-dd}/{file_uuid}.{ext}`
  - File Lifecycle:
    - Temporary storage for processing: 24 hours
    - Standard storage: 90 days
    - Archive storage: Beyond 90 days
- **Security:**
  - Server-side encryption with AES-256
  - Signed URLs for time-limited access
  - Object lifecycle policies

### 2.4.3 Search Indexing

- **Implementation:**
  - Elasticsearch for full-text search
  - Index Configuration:
    - Analyzer: language-specific (English default)
    - Tokenization: standard with word-boundary detection
    - Stemming: light stemming for English
    - Stop words: configurable, default English set
  - Index Strategy:
    - Document content and metadata



- Message content and metadata
- Real-time indexing on document/message updates

## 3. AI-Powered Contextualization Specifications

### 3.1 Natural Language Processing Pipeline

#### 3.1.1 Text Processing

- **Preprocessing:**
  - Tokenization: spaCy tokenizer with custom extensions
  - Sentence segmentation: rule-based with ML validation
  - Normalization: Case folding, Unicode normalization, contractions expansion
- **Feature Extraction:**
  - Language detection (threshold: 97% confidence)
  - Part-of-speech tagging
  - Named entity recognition (custom model for project-specific entities)
  - Dependency parsing for relationship extraction

#### 3.1.2 Keyword Extraction

- **Implementation:**
  - TF-IDF scoring with custom weighting
  - TextRank algorithm for extraction
  - N-gram extraction (n=1,2,3)
- **Parameters:**
  - Top-K cutoff: Configurable, default 20 keywords per document
  - Minimum occurrence: 2
  - Stop word filtering: Dynamic stop word list based on project context

#### 3.1.3 Topic Identification

- **Implementation:**
  - LDA (Latent Dirichlet Allocation) for unsupervised topic modeling
  - BERTopic for transformer-based topic detection
- **Parameters:**
  - Topic count: Automatically determined with coherence score optimization
  - Topic coherence threshold: 0.3 minimum
  - Topic granularity: Adjustable (1-5 scale, default 3)

#### 3.1.4 Entity Recognition

- **Implementation:**
  - Custom NER model trained on project data

- Entity types: PERSON, ORGANIZATION, LOCATION, DATE, PROJECT, PRODUCT, TECHNICAL\_TERM
- **Entity Linking:**
  - Coreference resolution to link entities across documents
  - Identity resolution to map entities to known objects
  - Confidence scoring for entity matches (threshold: 0.75)

## 3.2 Context Graph Generation

### 3.2.1 Graph Structure

- **Node Types:**
  - Document nodes
  - Message nodes
  - Entity nodes
  - Topic nodes
  - User nodes
- **Edge Types:**
  - Contains (Document -> Entity)
  - MentionedIn (Entity -> Document)
  - Discusses (Document -> Topic)
  - AuthoredBy (Document -> User)
  - Replied (Message -> Message)
  - Related (Document -> Document with weight attribute)

### 3.2.2 Relationship Discovery

- **Algorithm:**
  - Knowledge graph embedding with TransE model
  - Cosine similarity between document vectors (threshold: 0.6)
  - Entity co-occurrence analysis
- **Parameters:**
  - Minimum relationship strength: 0.4
  - Maximum connections per node: 15
  - Time decay factor: 0.9 per month

### 3.2.3 Graph Visualization

- **Implementation:**
  - D3.js force-directed graph
  - WebGL rendering for large graphs (>1000 nodes)
- **Visualization Features:**
  - Node sizing by importance
  - Edge thickness by relationship strength
  - Color coding by node type
  - Clustering by modularity

- Interactive zoom, pan, and filter
- Hover information cards
- Click-through to source content

### **3.3 Document Summarization**

#### **3.3.1 Summarization Techniques**

- **Extractive Summarization:**
  - TextRank algorithm implementation
  - Sentence importance scoring based on centrality
  - Length control: configurable, default 20% of original
- **Abstractive Summarization:**
  - Transformer-based summarization model
  - Fine-tuned on project-specific data
  - Length control: 3 settings (brief, standard, detailed)

#### **3.3.2 Key Points Extraction**

- **Implementation:**
  - Sentence classification for key point identification
  - Clustering of semantically similar sentences
  - Ranking by centrality and information content
- **Output:**
  - 3-5 key points per document (configurable)
  - Confidence score per key point
  - Source sentence reference

#### **3.3.3 Multi-Document Summarization**

- **Implementation:**
  - Cross-document coreference resolution
  - Redundancy elimination through MMR (Maximal Marginal Relevance)
  - Temporal ordering of information
- **Parameters:**
  - Maximum document count: 10
  - Diversity parameter: 0.7 (balance between relevance and diversity)
  - Length: Configurable, default 500 words

## **4. Collaborative Workspace Specifications**

### **4.1 Workspace Management**

#### **4.1.1 Workspace Creation and Configuration**

- **Features:**
  - Workspace name, description, and icon
  - Privacy settings (public, team, private)
  - Member invitation and management
  - Integration with data sources
- **Technical Implementation:**
  - Workspace object stored in Firestore
  - Workspace settings UI with real-time update
  - Bulk invitation via email or CSV upload

#### **4.1.2 Workspace Navigation**

- **Implementation:**
  - Hierarchical navigation structure
  - Quick access sidebar with pinned items
  - Recent items tracking (last 20 items)
  - Search within workspace
- **UI Components:**
  - Breadcrumb navigation
  - Tabbed interface for multiple open items
  - Split-view mode for document comparison

### **4.2 Document Management**

#### **4.2.1 Document Handling**

- **Features:**
  - Upload (drag-and-drop, file picker)
  - Create new documents (rich text editor)
  - Import from connected sources
  - Export to common formats
- **Technical Implementation:**
  - Progressive file upload with resumability
  - Client-side file validation
  - Server-side virus scanning
  - Format conversion microservice

#### **4.2.2 Version Control**

- **Implementation:**
  - Git-inspired versioning system
  - Full document history with diff visualization
  - Branching capability for collaborative editing
  - Conflict resolution UI
- **Technical Details:**
  - Delta-based storage for efficient versioning

- Automatic version creation on significant changes
- Manual version creation with annotations
- Version comparison with highlighting

#### 4.2.3 Document Preview

- **Supported Formats:**
  - Text: TXT, MD, RTF, DOC, DOCX, PDF
  - Spreadsheets: XLS, XLSX, CSV
  - Presentations: PPT, PPTX
  - Images: JPG, PNG, GIF, SVG
  - Code: Various programming languages with syntax highlighting
- **Implementation:**
  - Server-side rendering for consistent display
  - Client-side preview for quick access
  - Thumbnail generation for visual browsing
  - Mobile-optimized viewing experience

### 4.3 Discussion System

#### 4.3.1 Thread Management

- **Features:**
  - Create discussion threads
  - Reply to threads
  - Thread categorization and tagging
  - Thread status (open, resolved, archived)
- **Technical Implementation:**
  - Threaded data model in Firestore
  - Real-time updates using WebSockets
  - Pagination with cursor-based approach
  - Thread subscription and notification

#### 4.3.2 Rich Communication

- **Features:**
  - Rich text formatting (Markdown support)
  - @mentions with user autocomplete
  - Document/message references
  - Code formatting with syntax highlighting
- **Implementation:**
  - Rich text editor (ProseMirror)
  - Mention plugin with user directory integration
  - Link unfurling for referenced documents
  - Emoji picker and reaction system

### 4.3.3 File Attachments

- **Features:**
  - Attach files to discussion posts
  - Drag-and-drop upload
  - Image preview in-line
  - File versioning
- **Implementation:**
  - Client-side image compression
  - Progressive upload with progress indicator
  - Preview generation for common formats
  - Storage integration with permission inheritance

### 4.3.4 Notification System

- **Features:**
  - In-app notifications
  - Email digests (configurable frequency)
  - Browser push notifications
  - Mobile push notifications
- **Implementation:**
  - Notification preferences by workspace and thread
  - Batching logic to prevent notification spam
  - Read status tracking
  - Smart notification routing based on user activity

## 5. Intelligent Search and Discovery Specifications

### 5.1 Search Infrastructure

#### 5.1.1 Search Engine

- **Technology:** Elasticsearch 8.x
- **Deployment:**
  - Minimum 3-node cluster for redundancy
  - Automated index management
  - Rolling updates with zero downtime
- **Scaling:**
  - Horizontal scaling with automatic sharding
  - Shard allocation awareness for multi-zone deployment
  - Resource-based auto-scaling

#### 5.1.2 Index Configuration

- **Index Strategy:**

- One index per tenant
- Daily rolling indices for logs and analytics
- Aliases for simplified access
- **Mapping:**
  - Dynamic mapping with templates
  - Custom analyzers for specialized fields
  - Field types optimized for search performance
- **Analyzers:**
  - Standard analyzer with custom filters
  - Language-specific analyzers (English, Spanish, French, German, Japanese)
  - N-gram analyzer for partial matching
  - Edge n-gram analyzer for autocomplete

## 5.2 Search Functionality

### 5.2.1 Query Processing

- **Query Types:**
  - Full-text search
  - Phrase search
  - Fuzzy search (Levenshtein distance: 2)
  - Prefix search
  - Boolean search (AND, OR, NOT operators)
- **Implementation:**
  - Query string parsing
  - Query expansion with synonyms
  - Query relaxation for zero-result scenarios
  - Query highlighting with context

### 5.2.2 Advanced Search Features

- **Semantic Search:**
  - Vector-based search using document embeddings
  - Query expansion using word embeddings
  - Relevance tuning with machine learning
- **Faceted Search:**
  - Dynamic facet generation
  - Multi-select facets
  - Hierarchical facets
  - Histogram facets for numeric/date fields
- **Filters:**
  - Document type filter
  - Date range filter
  - Source platform filter
  - Author filter

- Content type filter (document, message, comment)

### **5.2.3 Results Presentation**

- **Ranking:**
  - BM25 algorithm with custom boosting
  - Recency boost (configurable weight)
  - Popularity signal integration
  - User-specific personalization
- **Results UI:**
  - Snippet generation with highlighted matches
  - Result grouping by type
  - Infinite scroll with pagination
  - Result expansion for preview
  - Quick actions from results

## **5.3 Discovery Features**

### **5.3.1 Personalized Recommendations**

- **Algorithms:**
  - Collaborative filtering for document recommendations
  - Content-based filtering using document similarity
  - Hybrid approach combining both signals
- **Implementation:**
  - User activity tracking (views, edits, comments)
  - Document feature extraction
  - Recommendation generation job (hourly)
  - Recommendation serving API

### **5.3.2 Trending Content**

- **Detection:**
  - Activity spike detection
  - Time-decay scoring
  - Workspace-specific trending algorithms
- **Presentation:**
  - Daily trending items
  - Weekly digest
  - Topic-based trending
  - Visual trending indicators

### **5.3.3 Related Content Suggestions**

- **Implementation:**
  - Content similarity calculation



- Co-access patterns
- Temporal proximity
- Author-based relationships
- **Integration:**
  - Sidebar suggestions while viewing content
  - "You might also like" section
  - Contextual links within content
  - Discovery feed personalization

## 6. Role-Based Access and Permissions Specifications

### 6.1 Authentication and Authorization Framework

#### 6.1.1 Authentication System

- **Methods:**
  - Email/password with PBKDF2 hashing
  - Single Sign-On (SSO):
    - SAML 2.0
    - OpenID Connect
    - OAuth 2.0
  - Multi-factor authentication (TOTP)
- **Session Management:**
  - JWT token-based sessions
  - Configurable session duration
  - Session invalidation API
  - Device tracking and management

#### 6.1.2 Authorization Model

- **Framework:**
  - Role-Based Access Control (RBAC)
  - Attribute-Based Access Control (ABAC) extensions
  - Policy enforcement points at API and UI levels
- **Implementation:**
  - Centralized policy service
  - Permission evaluation cache
  - Policy auditing and logging
  - Policy version control

## 6.2 Role Management

### 6.2.1 Role Definition

- **System Roles:**
  - Super Admin: Complete system access
  - Admin: Organizational administration
  - Member: Standard user with customizable permissions
  - Viewer: Read-only access
  - Guest: Limited temporary access
- **Custom Roles:**
  - Role creation UI
  - Permission selection interface
  - Role hierarchy configuration
  - Role cloning functionality

### **6.2.2 Permission Management**

- **Permission Types:**
  - Resource-level permissions (create, read, update, delete)
  - Feature-level permissions (search, export, invite)
  - Data-level permissions (specific document types)
- **Permission Assignment:**
  - Direct assignment to users
  - Role-based assignment
  - Group-based assignment
  - Temporary permission grants

## **6.3 Access Control Implementation**

### **6.3.1 Project-Level Access**

- **Access Models:**
  - Private: Explicit access required
  - Team: Automatic access to team members
  - Public: Accessible to all authenticated users
- **Implementation:**
  - Access control lists (ACLs)
  - Inheritance from parent projects
  - Override capability at any level
  - Access request workflow

### **6.3.2 Document-Level Access**

- **Permission Types:**
  - View: Read-only access
  - Comment: View + ability to add comments
  - Edit: Comment + ability to modify content
  - Manage: Edit + ability to change permissions
  - Owner: Full control

- **Implementation:**
  - Document-specific ACLs
  - Inheritance from project settings
  - Share links with configurable permissions
  - Expiring access grants

### **6.3.3 Data Security Controls**

- **Data Segregation:**
  - Tenant isolation at database level
  - Row-level security in PostgreSQL
  - Collection-level security in Firestore
- **Encryption:**
  - Data-at-rest encryption (AES-256)
  - Transport encryption (TLS 1.3)
  - Field-level encryption for sensitive data
- **Audit Trail:**
  - Comprehensive access logging
  - Permission change history
  - Export and reporting capabilities
  - Anomaly detection

## **7. System Integration Specifications**

### **7.1 API Framework**

#### **7.1.1 GraphQL API**

- **Implementation:**
  - Apollo Server with Express
  - Schema-first design approach
  - Type definitions with strong typing
  - Resolver implementation with dataloader pattern
- **Features:**
  - Query batching and caching
  - Subscription support via WebSockets
  - Partial response support
  - Query complexity analysis

#### **7.1.2 REST Endpoints**

- **Implementation:**
  - Express.js REST controllers
  - OpenAPI 3.0 specification
  - Versioned endpoints (/api/v1/\*)

- Standard HTTP methods and status codes
- **Features:**
  - HATEOAS links for navigation
  - ETag support for caching
  - Bulk operations
  - Rate limiting

## 7.2 Webhook System

### 7.2.1 Outgoing Webhooks

- **Events:**
  - Document created/updated/deleted
  - Message created/updated
  - User actions (login, permission changes)
  - System events (integration status)
- **Implementation:**
  - Event queue with RabbitMQ
  - Webhook delivery service
  - Retry mechanism (exponential backoff)
  - Signature verification (HMAC-SHA256)

### 7.2.2 Incoming Webhooks

- **Capabilities:**
  - Data ingestion from external systems
  - Command execution (create document, update, etc.)
  - Integration event handling
- **Security:**
  - Token-based authentication
  - Request verification
  - Rate limiting
  - IP whitelisting

## 7.3 External Integrations

### 7.3.1 Third-Party Services

- **Authentication Providers:**
  - Google
  - Microsoft
  - Okta
  - Auth0
- **Storage Providers:**
  - Google Drive
  - Dropbox

- OneDrive
  - Box
- **Communication Services:**
  - Slack
  - Microsoft Teams
  - Discord
  - Zoom

### 7.3.2 Integration SDK

- **Features:**
  - Client libraries (JavaScript, Python, Ruby)
  - Authentication helpers
  - Data models
  - Example code
- **Documentation:**
  - API reference
  - Integration guides
  - Sample applications
  - Troubleshooting guides

## 8. Performance and Scalability Specifications

### 8.1 Performance Requirements

#### 8.1.1 Response Time Targets

- **Web UI:**
  - Page load: < 2 seconds
  - Interactive response: < 200ms
  - Search results: < 500ms
- **API:**
  - Simple queries: < 100ms
  - Complex queries: < 500ms
  - Batch operations: < 2 seconds per 100 items

#### 8.1.2 Throughput Requirements

- **Concurrent Users:**
  - Small tenant: 50 concurrent users
  - Medium tenant: 250 concurrent users
  - Large tenant: 1000+ concurrent users
- **Operations:**
  - Read operations: 1000/second
  - Write operations: 100/second

- Search queries: 50/second

## **8.2 Scalability Architecture**

### **8.2.1 Horizontal Scaling**

- **Services:**
  - Stateless microservices architecture
  - Containerization with Docker
  - Orchestration with Kubernetes
  - Auto-scaling based on CPU/memory metrics
- **Database:**
  - Firestore: Global replication
  - PostgreSQL: Read replicas and sharding
  - Elasticsearch: Cluster scaling

### **8.2.2 Caching Strategy**

- **Layers:**
  - Browser caching (HTTP Cache-Control)
  - CDN caching for static assets
  - Application caching (Redis)
  - Database query cache
- **Implementation:**
  - Cache invalidation based on write operations
  - Time-to-live (TTL) settings by data type
  - Cache warming for popular content
  - Cache hit rate monitoring

## **8.3 Data Volume Management**

### **8.3.1 Storage Planning**

- **Capacity:**
  - Initial storage: 100GB per tenant
  - Growth projection: 5-10GB per month per active tenant
  - Archival strategy: Cold storage after 12 months
- **Content Limits:**
  - Maximum file size: 100MB
  - Maximum attachment count: 50 per document
  - Maximum document length: 10MB text content

### **8.3.2 Data Retention**

- **Policies:**
  - Active data: 12 months

- Archived data: 7 years
- Deletion logs: Permanent
- **Implementation:**
  - Automated archival process
  - Data export before deletion
  - Compliance with regulatory requirements
  - User-configurable retention policies

## 9. Security Specifications

### 9.1 Security Standards

#### 9.1.1 Compliance Requirements

- **Frameworks:**
  - SOC 2 Type II
  - GDPR
  - CCPA
  - ISO 27001
- **Industry Standards:**
  - OWASP Top 10 mitigation
  - NIST Cybersecurity Framework
  - CIS Benchmarks

#### 9.1.2 Security Testing

- **Methods:**
  - Static Application Security Testing (SAST)
  - Dynamic Application Security Testing (DAST)
  - Penetration testing (quarterly)
  - Vulnerability scanning (weekly)
- **Process:**
  - Pre-commit hooks for security scanning
  - CI/CD pipeline security checks
  - Bug bounty program
  - Third-party security audits

### 9.2 Data Protection 9.2.1 Encryption

- **In-Transit:**
  - TLS 1.3 for all communications
  - Perfect Forward Secrecy
  - Strong cipher suites (ECDHE-RSA-AES256-GCM-SHA384)
  - Certificate pinning for mobile applications
  - API request signing for additional verification

- **At-Rest:**
  - AES-256 encryption for all stored data
  - Database-level encryption
  - File-level encryption for documents and attachments
  - Encryption key management using a dedicated HSM (Hardware Security Module)
  - Regular key rotation policy (quarterly)
- **End-to-End:**
  - Zero-knowledge architecture for sensitive communications
  - Client-side encryption of personal/confidential data
  - Secure key exchange protocols
  - No backdoor access to encrypted content

#### 9.2.2 Access Controls

- Role-based access control (RBAC)
- Principle of least privilege implementation
- Multi-factor authentication for all administrative access
- Privileged access management (PAM)
- Regular access reviews and certifications

#### 9.2.3 Data Retention and Disposal

- Automated data retention policies based on data classification
- Secure deletion procedures for expired data
- Regular audits of stored data to identify and remove unnecessary information
- Dedicated processes for handling data subject deletion requests
- Certificate of destruction provided for sensitive data disposal
- Secure hardware decommissioning procedures with documented chain of custody

#### 9.2.4 Data Loss Prevention (DLP)

- Comprehensive DLP solution monitoring all data exit points
- Content inspection for sensitive information in transit
- Automated blocking of unauthorized data transfers
- Regular DLP policy updates based on emerging threats
- User activity monitoring for suspicious data access patterns
- Incident response procedures for potential data leaks

#### 9.2.5 Backup and Recovery

- Daily incremental and weekly full backups
- Encrypted backup storage
- Geographically dispersed backup locations
- Regular recovery testing (quarterly)
- Point-in-time recovery capabilities
- Documented recovery time objectives (RTO) and recovery point objectives (RPO)
- Automated backup verification and integrity checks



#### 9.2.6 Vendor Management

- Third-party security assessment process
- Data processing agreements with all vendors
- Regular security reviews of key service providers
- Contractual obligations for data protection
- Vendor breach notification requirements
- Right to audit clauses in all contracts involving personal data

#### 9.2.7 Privacy by Design

- Privacy impact assessments for new projects
- Data minimization principles in product development
- Anonymization and pseudonymization techniques
- User-controlled privacy settings
- Transparent data collection practices
- Built-in consent management

#### 9.2.8 Data Breach Response

- Dedicated incident response team
- Documented data breach notification procedures
- Internal escalation protocols with defined timelines
- External communication templates and processes
- Regular breach simulation exercises
- Post-incident analysis and remediation planning
- Compliance with regulatory reporting requirements
- Evidence preservation methodology

#### 9.2.9 Data Classification

- Tiered classification system (Public, Internal, Confidential, Restricted)
- Automated data discovery and classification tools
- Classification-based security controls
- Regular classification reviews
- User training on proper data handling by classification
- Visual marking of sensitive documents
- Metadata tagging for digital assets

#### 9.2.10 Cross-Border Data Transfers

- Data transfer impact assessments
- Standard contractual clauses (SCCs) implementation
- Binding corporate rules for intra-group transfers
- Regional data residency options
- Documentation of international data flows
- Regular monitoring of data transfer regulations
- Data sovereignty compliance checks

### 9.2.11 Secure Development Practices

- Secure coding standards
- Regular security testing in the development lifecycle
- Vulnerability scanning of applications
- Code reviews focused on security
- Application security training for developers
- Security requirements integrated into user stories
- Third-party component vulnerability management

### 9.2.12 Compliance Monitoring

- Automated compliance scanning
- Regular internal audits
- External compliance assessments
- Compliance dashboard with key metrics
- Remediation tracking for identified issues
- Regulatory change monitoring
- Evidence collection and maintenance

## 9.3 Authentication and Authorization 9.3.1 User Authentication

- Multi-factor authentication support
- Single sign-on (SSO) integration
- Biometric authentication options
- Progressive authentication based on risk assessment
- Secure password policies and hashing
- Account lockout protection against brute force attacks

### 9.3.2 Session Management

- Secure session handling
- Automatic session timeouts
- Concurrent session limitations
- Session revocation capabilities
- Device tracking and management

## 10. Compliance Requirements

### 10.1 Regulatory Compliance

- GDPR compliance measures
- CCPA/CPRA compliance
- HIPAA compliance (if applicable)
- PCI DSS compliance (if handling payment data)
- SOC 2 compliance controls
- Industry-specific regulations

### 10.2 Audit Capabilities

- Comprehensive audit logging
- Tamper-proof audit trails
- Audit log retention policies
- Audit log search and analysis tools
- Automated suspicious activity reporting

### 10.3 Certifications

- Required security certifications
- Compliance attestations
- Third-party audit requirements
- Annual recertification processes

## 11. Performance Requirements

### 11.1 Scalability

- Expected user growth projections
- Peak load handling capabilities
- Horizontal and vertical scaling strategies
- Resource utilization targets
- Dynamic resource allocation

### 11.2 Availability

- Uptime requirements (99.9%+)
- Planned maintenance windows
- Redundancy requirements
- Fault tolerance capabilities
- Geographic distribution strategy

### 11.3 Response Time

- Maximum acceptable latency
- Average response time targets
- Transaction processing speed requirements
- Rendering time expectations
- API response time SLAs

### 11.4 Capacity

- Concurrent user support
- Data storage requirements
- Bandwidth provisions
- Transaction volume handling
- Growth projections (12, 24, 36 months)

## 12. Integration Requirements

### 12.1 External Systems

- Required third-party integrations
- API specifications for external systems
- Authentication requirements for external services
- Data exchange formats and protocols
- Fallback procedures for integration failures

## 12.2 Internal Systems

- Enterprise system integration points
- Data synchronization requirements
- Single source of truth determinations
- Integration testing requirements

## 13. User Experience Requirements

### 13.1 Accessibility

- WCAG 2.1 AA compliance
- Screen reader compatibility
- Keyboard navigation support
- Color contrast requirements
- Accessibility testing procedures

### 13.2 Internationalization

- Supported languages
- Localization requirements
- Date, time, and number format handling
- Right-to-left language support
- Cultural considerations

### 13.3 Design Standards

- Brand compliance requirements
- Design system adherence
- UI component library usage
- Consistency requirements
- Mobile-first design principles

## 14. Deployment and Support

### 14.1 Deployment Requirements

- Deployment model (on-premises, cloud, hybrid)
- Environment specifications
- Deployment frequency expectations
- Zero-downtime deployment requirements
- Rollback capabilities

### 14.2 Monitoring and Alerting

- System health monitoring
- Performance monitoring
- User behavior analytics
- Proactive alerting thresholds
- Incident classification criteria

### 14.3 Support Model

- Support hours and SLAs
- Tier structure for support
- Escalation procedures
- Knowledge base requirements
- Self-service support tools

## 15. Documentation Requirements

### 15.1 User Documentation

- End-user guides
- Administrator manuals
- Video tutorials
- Contextual help content
- FAQ requirements

### 15.2 Technical Documentation

- System architecture documentation
- API documentation
- Database schema documentation
- Development guides
- Operations runbooks

## 16. Success Metrics

### 16.1 Business Metrics

- Key performance indicators
- Revenue targets
- User adoption goals
- Retention metrics
- Conversion rates

### 16.2 Technical Metrics

- Performance benchmarks
- Error rate thresholds
- System reliability metrics
- Security incident measurements
- Technical debt management

## 17. Appendices

## 17.1 Glossary

- Definition of key terms
- Acronyms and abbreviations
- Industry-specific terminology

## 17.2 References

- Related documents
- Regulatory standards
- Industry best practices
- Research sources

## 17.3 Revision History

- Document version control
- Change log
- Approval history
- Distribution list